is the case and a sample implementation of the equality comparison operator.

Listing 3.10. Locking one mutex at a time in a comparison operator

```
class Y
{
private:
    int some_detail;
    mutable std::mutex m;

    int get_detail() const
    {
        std::lock_guard<std::mutex> lock_a(m);      ❶
        return some_detail;
    }
public:
    Y(int sd):some_detail(sd){}

    friend bool operator==(Y const& lhs, Y const& rhs)
    {
        if(&lhs==&rhs)
            return true;
        int const lhs_value=lhs.get_detail();       ❷
        int const rhs_value=rhs.get_detail();       ❸
        return lhs_value==rhs_value;                ❹
    }
};
```

In this case, the comparison operator first retrieves the values to be compared by calling the `get_detail()` member function ❷, ❸. This function retrieves the value while protecting it with a lock ❶. The comparison operator then compares the retrieved values ❹. Note, however, that as well as reducing the locking periods so that only one lock is held at a time (and thus eliminating the possibility of deadlock), *this has subtly changed the semantics of the operation* compared to holding both locks together. In listing 3.10, if the operator returns `true`, it means that the value of `lhs.some_detail` at one point in time is equal to the value of `rhs.some_detail` at another point in time. The two values could have been changed in any way in between the two reads; the values could have been swapped in between ❷ and ❸, for example, thus rendering the comparison meaningless. The equality comparison might thus return `true` to indicate that the values were equal, even though there was never an instant in time when the values were actually equal. It’s therefore important